

Smart Energy Meter

Complete system guide: architecture, roles and permissions, OTP security, ESP32 firmware, API reference, setup and troubleshooting.

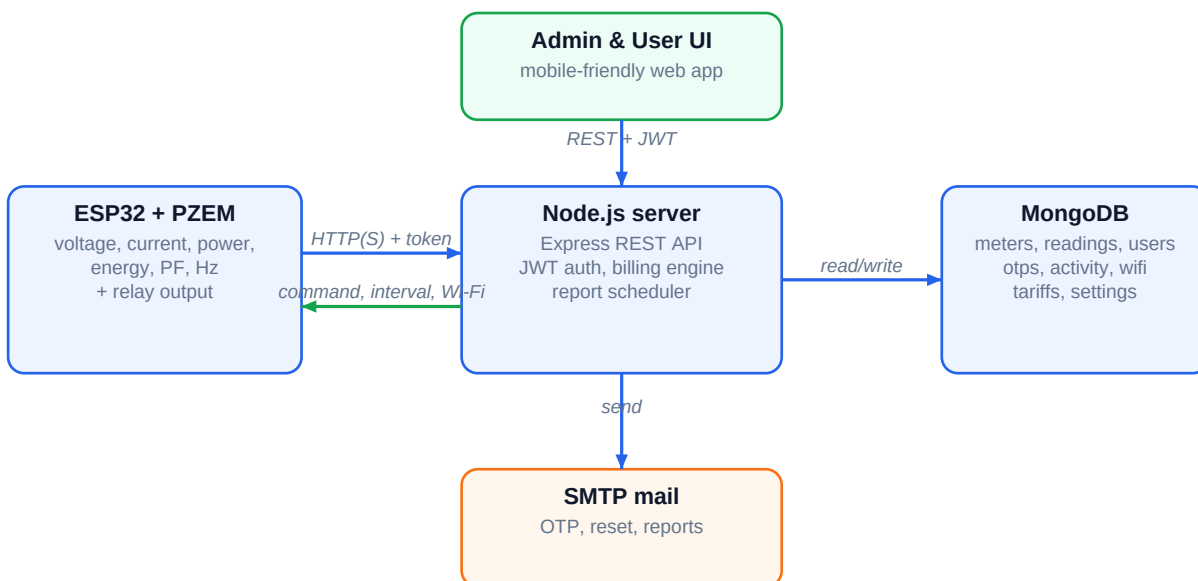
Version 3.0 | Node.js + MongoDB + ESP32/PZEM

1. What the system does

Smart Energy Meter reads electrical values from a PZEM-004T sensor on an ESP32, sends them to a Node.js server over HTTP(S), stores every reading in MongoDB, converts usage into an estimated bill using admin-defined tariff slabs, and lets an **Admin** and **Users** watch and control the meter from a phone-friendly web app.

Capability	How it works
Live monitoring	Voltage, current, power, energy, power factor, frequency, Wi-Fi signal and online duration, refreshed every 3-5 s.
Billing	Progressive slabs, FAC, duty, wheeling, fixed and other charges, billing-cycle day, projected bill, budget progress.
Relay control	Admin always; user only if the admin enables User may control relay for that meter.
Meter configuration	If enabled by admin, the assigned user renames the meter, changes interval and pushes new Wi-Fi remotely.
OTP security	Emailed 6-digit codes for login and password reset, every code logged in the otps table.
Reports and alerts	Daily/weekly/monthly emailed reports, on-demand user reports, threshold alerts and saving tips.
Audit and export	90-day activity log; CSV export of raw readings.

Architecture

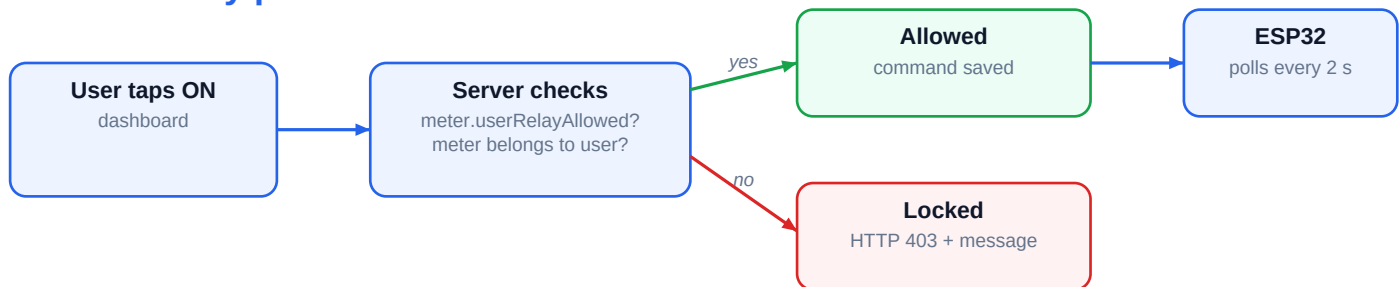


The ESP32 never accepts inbound connections. It only calls the server, authenticated by a per-meter secret token, so it works behind any home router with no port forwarding.

2. Roles and permissions

Action	Admin	User (own meter)
See live readings, usage, bill estimate, alerts	All meters	Own meters
Register / delete meters, rotate device token	Yes	No
Assign a meter to a user	Yes	No
Switch relay ON/OFF	Always	Only if userRelayAllowed (default: locked)
Rename meter, change interval, scan/connect Wi-Fi	Yes	Only if userConfigAllowed (default: allowed)
Tariff slabs, charges, limits, report schedule, SMTP	Edit	View only (shown under <i>My meter</i>)
Create, disable, delete users; set a password	Yes	No
Enable OTP login; view OTP table and activity log	Yes	No
Change own password; request an emailed report; CSV	Yes	Yes

How the relay permission is enforced

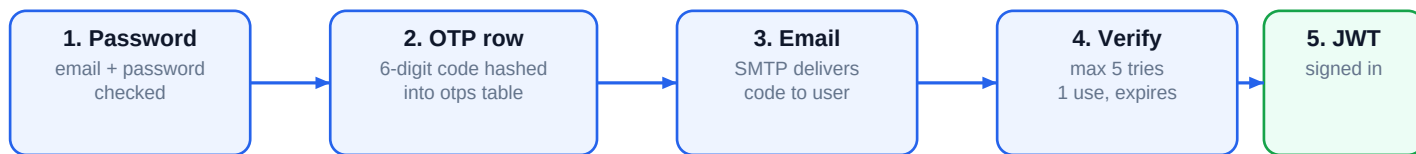


The check lives in the server route (PUT `/api/user/meters/:id/command`), so editing the page in a browser cannot bypass it. The user dashboard additionally hides the buttons and shows a lock notice when the permission is off. The same pattern guards `userConfigAllowed` for settings and Wi-Fi routes.

Per-meter switches (admin: Meters & control)

- **User may control relay** - toggles `userRelayAllowed`.
- **User may configure meter** - toggles `userConfigAllowed`.
- Every change is written to the activity log with the admin's name.

3. OTP and account security



Every code is stored as a row in the **otps** collection (the OTP table). The plain code is never stored - only a SHA-256 hash bound to the user and purpose.

Field	Meaning
userId / email	Who the code was issued to.
purpose	login (second step after password) or reset (forgot password).
codeHash	SHA-256 of userId:purpose:code. Compared in constant time.
expiresAt	Now + validity (2-30 min, default 10). Rows are auto-deleted 24 h after expiry.
attempts	Wrong guesses; the code dies after 5.
usedAt	Set on success - a code works once.
delivery / ip	Whether SMTP accepted it, and the requester's IP.

Rules built in

- A new code cancels any earlier unused code for the same user and purpose.
- Resend limit: one code per 60 seconds per user and purpose.
- Forgot-password always answers with the same message, so attackers cannot discover which emails exist.
- Admins can require OTP for **users**, **admins**, or both. Enabling it requires SMTP to be configured, and for admins a successful test email first, so you cannot lock yourself out.
- The admin *OTP & security* page shows the latest 100 rows with state: ACTIVE, USED, EXPIRED or FAILED.

Other protections

- Passwords hashed with bcrypt; JWT sessions last 7 days; disabled users are rejected on the next request.
- SMTP and Wi-Fi passwords are encrypted with AES-256-GCM (keys SMTP_CONFIG_KEY, WIFI_CONFIG_KEY in .env).
- Device tokens are stored only as SHA-256 hashes and shown once at pairing.

4. ESP32 firmware (nothing hard-coded)

File: ESP32_FINAL.ino. Libraries: WiFiManager, PZEM-004T v30, ArduinoJson v7; board package ESP32 Arduino core 3.x.

First-time setup

- Flash the sketch. With empty settings the ESP32 opens the hotspot **ENERGY-METER-SETUP**.
- Join it from a phone and open 192.168.4.1.
- Pick your Wi-Fi and fill the fields below, then Save. The values are stored in flash and reused on every boot.

Portal field	Example	Notes
Server URL	http://192.168.1.10:5000	http or https; trailing slash removed automatically.
Meter ID	METER01	Must match the ID registered by the admin.
Device token	64 hex chars	Shown once when the admin registers or rotates.
Relay GPIO pin	5	Change to match your wiring - no re-flash needed.
Relay active level	1 = LOW triggers	0 for modules that switch on HIGH.
PZEM RX / TX pins	16 / 17	ESP32 RX connects to PZEM TX.

Runtime behaviour

- **Single poll every 2 s** returns relay command, reading interval and any Wi-Fi job; readings are posted at the admin/user-set interval (1-3600 s).
- **Fail-safe:** relay is OFF at boot until the server states the wanted state.
- **Remote Wi-Fi change:** dashboard scan list comes from the ESP32 itself. If the new network fails in 20 s the old one is restored and the failure is reported.
- **Watchdog and recovery:** 60 s hardware watchdog, automatic Wi-Fi reconnect, restart after 20 consecutive failed uploads.
- **Factory reset:** hold BOOT (GPIO 0) for 5 s - settings and Wi-Fi are wiped and the portal returns.
- Missing PZEM values are sent as null, never as zero, so charts show gaps instead of false data.

Wiring reminder: mains work must be done by a qualified person. Use the PZEM and relay manufacturer wiring diagrams.

5. Data model (MongoDB)

Collection	Purpose	Key fields
users	Accounts	name, email, password hash, role, active
meters	Registered devices and latest values	meterId, userId, command, status, userRelayAllowed, userConfigAllowed, updateFrequency, rssi, firmware, lastSeen
readings	Time series of every report	meterId, voltage, current, power, energy, frequency, powerFactor, createdAt
otps	OTP table	userId, purpose, codeHash, expiresAt, attempts, usedAt, delivery, ip
activitylogs	Audit trail (90 days)	actorName, action, target, detail, createdAt
appsettings	Security switches	otpForUsers, otpForAdmins, otpTtlMinutes
tariffslabs / billingsettings	Pricing, limits	slabs; cycle day, charges, thresholds, alerts
wifiprovisionings	Remote Wi-Fi jobs	scan result, encrypted password, revision, status
smtpsettings / reportsettings	Mail account, report schedule	encrypted SMTP password; daily/weekly/monthly, hour

6. REST API summary

Endpoint	Who	Purpose
POST /api/auth/login	public	Password step; returns token or <i>otpRequired</i> .
POST /api/auth/verify-otp, /resend-otp	public	Complete or repeat the OTP step.
POST /api/auth/forgot-password, /reset-password	public	OTP-based password reset.
PUT /api/auth/change-password	any user	Change own password.
GET /api/user/meters	user	Own meters incl. permission flags.
PUT /api/user/meters/:id/command	user*	Relay ON/OFF (*needs userRelayAllowed).
PUT /api/user/meters/:id/settings	user*	Name and interval (*needs userConfigAllowed).
GET/POST /api/user/meters/:id/wifi[/scan[/connect]]	user*	Remote Wi-Fi (*needs userConfigAllowed).
GET /api/user/assigned-config	user	Read-only tariffs, limits, schedule, permissions.
GET /api/meters/:id/consumption, /history, /export.csv	owner/admin	Analytics, raw history, CSV.
PUT /api/admin/meters/:id/permissions	admin	Set userRelayAllowed / userConfigAllowed.
POST/DELETE /api/admin/meters ...	admin	Register, pair, rename, assign, delete meters.
GET/PUT /api/admin/users ...	admin	Create, disable, delete, set password.
GET/PUT /api/admin/security, GET /api/admin/otps	admin	OTP policy and OTP table.
GET /api/admin/activity	admin	Audit log.
POST /api/meter/data, GET /api/device/:id/settings	device token	ESP32 upload and poll.

7. Admin and user pages

Both apps share one design system with automatic dark mode (and a manual toggle), 44 px touch targets, and no horizontal scrolling on phones.

Area	Desktop	Phone
Admin navigation	Grouped sidebar	Slide-in drawer opened by the menu button
User navigation	Segmented tab bar	Fixed bottom tab bar: Overview, Usage, My meter, Insights, Account
Meter lists	Responsive card grid	Single column cards with toggle switches
Tables (users, OTP, activity)	Scrollable tables	Each row becomes a labelled card
Forms	3-column grid	Single column, large inputs

Admin pages

- **Dashboard** - fleet totals and live cards. **Consumption** - charts, bill breakdown, CSV.
- **Meters & control** - register, assign, relay, interval, permission switches, token, rename, delete.
- **ESP32 Wi-Fi, Users, Tariff slabs, Billing & alerts, Email reports, Mail delivery.**
- **OTP & security** - policy toggles and the OTP table. **Activity log.**

User pages

- **Overview** - running cost, today/period/expected bill, budget bar, live readings.
- **Usage** - hourly/daily/weekly/monthly charts and CSV download.
- **My meter** - relay (if allowed), name/interval, Wi-Fi scan and connect (if allowed), plus everything assigned by the admin.
- **Insights** - alerts, saving suggestions, emailed reports. **Account** - change password.

8. Installation

Step	Command / action
1	Install Node.js 18+ and MongoDB.
2	Copy <code>.env.example</code> to <code>.env</code> ; set <code>MONGO_URI</code> , <code>JWT_SECRET</code> , <code>ADMIN_EMAIL</code> , <code>ADMIN_PASSWORD</code> (12+ chars), <code>SMTP_CONFIG_KEY</code> and <code>WIFI_CONFIG_KEY</code> (32+ random chars each).
3	<code>npm install</code> then <code>npm run seed-admin</code> then <code>npm start</code> .
4	Sign in as admin, open Mail delivery , save SMTP and send the test email.
5	Add tariff slabs and billing settings.
6	Meters & control - register a meter, copy the one-time token.
7	Flash the ESP32 and fill the setup portal (section 4).
8	Create users, assign meters, then flip the permission switches you want.
9	Optional: OTP & security - require OTP for users (and admins after a passing SMTP test).

Use HTTPS (reverse proxy such as Caddy or nginx) when the server is reachable from the internet. Set the ESP32 server URL to the https address.

9. Troubleshooting

Symptom	Check
Meter stays OFFLINE	Server URL reachable from the ESP32 network? Meter ID and token correct? Serial monitor shows HTTP code (401 = token rotated).
User gets 'relay not allowed'	Admin: Meters & control - turn on User may control relay .
No OTP email	Mail delivery - run the test email; check spam; rows with state FAILED in the OTP table mean SMTP rejected it.
Locked out after enabling admin OTP	Set <code>otpForAdmins: false</code> in the <code>appsettings</code> collection, or fix SMTP.
Wi-Fi scan list empty	Meter must be paired and online; wait one poll (2 s) after pressing Scan.
Wi-Fi change failed	Wrong password or out of range; the meter rolled back to its previous network.
Cost shows --	Add tariff slabs and wait for at least two energy readings in the period.
Relay inverted	Portal: flip <i>Relay active level</i> (1/0).

Costs are estimates based on configured tariffs and are not a utility bill.